

Team Project: Campus Event Planner Web Application

The following is a real-life project brief with Jira workflow and detailed app specification

1. Project Description

Your team will design, build, test, and deploy a **Campus Event Planner** web application. This app helps users discover, create, and manage events happening across the university campus.

The project simulates a real software development environment. You will:

- Work in a Scrum-style team
- Use **Jira** to manage your backlog, sprints, and workflow
- Use **Git + GitHub** for branching, pull requests, and code reviews
- Deploy your app to a **free cloud platform**
- Produce a working web application with a public URL

This project is intentionally scoped to be achievable in a few hours while.

2. Functional Requirements (Detailed)

Your app must include the following features.

2.1 Event List Page

- Displays all upcoming events
- Shows:
 - Event title
 - Date & time
 - Location
 - Description
- Events must be sorted by date
- Should update automatically when new events are added

2.2 Add Event Form

- Form fields:
 - Title (required)
 - Date (required)
 - Time (required)
 - Location (required)
 - Description (optional)
- Form validation:
 - Required fields cannot be empty
 - Date must be in the future

2.3 Edit/Delete Events

- Users can update event details
- Users can delete an event
- Confirmation dialog recommended

2.4 Data Storage Options

Choose one:

Option A: JSON File (simplest)

- Store events in a JSON file
- Use JavaScript to read/write (if allowed)

Option B: Lightweight Backend

- Node.js + Express
- Flask (Python)
- Stores events in memory or a JSON file

Option C: Hosted JSON API

- mockapi.io
- jsonbin.io
- Supabase free tier

This keeps the project realistic without requiring a full database.

2.5 Responsive UI

- Must work on mobile and desktop
- Use CSS Grid/Flexbox or a framework like Bootstrap/Tailwind

2.6 Deployment

Deploy to one of:

- **Netlify** (static or JS apps)
- **Vercel** (React/Next.js)
- **Render** (Node/Flask backend)

Your deployed app must be publicly accessible.

3. Optional Enhancements (Sprint 2)

Teams may add extra features:

- Event categories
- Search bar
- Dark mode
- Event images
- “Featured event” banner
- User login (mock only)
- Calendar view

These are not required but can boost marks.

4. Team Roles

Role	Responsibilities
Product Owner	Writes user stories, manages backlog, prioritises work
Scrum Master	Runs stand-ups, ensures workflow is followed
Developers	Build features, write tests, review PRs
DevOps Lead	Handles deployment, monitors build failures

Teams may rotate roles each sprint.

5. Jira: Deep-Dive Instructions

This section gives students a **realistic, industry-style Jira workflow**.

Step 1: Create the Jira Project

- Create a **Scrum** project
- Name it: *Campus Event Planner – Team X*
- Add all team members

Step 2: Create Epics

Epics are large bodies of work. Create these:

1. **EPIC 1: Event Browsing**
2. **EPIC 2: Event Management (CRUD)**
3. **EPIC 3: UI/UX Design**
4. **EPIC 4: Deployment & DevOps**
5. **EPIC 5: Testing & Quality Assurance**

Step 3: Write User Stories

Each story uses the format:

As a [user], I want [feature], so that [benefit].

Example User Stories (Add these to Jira)

Epic	User Story
Event Browsing	As a student, I want to view a list of events so I can see what's happening on campus.
Event Browsing	As a user, I want events sorted by date so I can quickly find upcoming ones.
Event Management	As an organiser, I want to add an event so others can attend.
Event Management	As a user, I want to edit or delete events so I can keep information accurate.
UI/UX	As a user, I want the app to work on mobile so I can check events on the go.
DevOps	As a developer, I want the app deployed online so users can access it.
QA	As a tester, I want to verify form validation so users can't submit incomplete events.

Step 4: Break Stories into Tasks

Each story becomes 2–6 tasks.

Example Breakdown

Story: "As a student, I want to view a list of events..."

Tasks:

- Create event list HTML structure
- Style event cards
- Fetch events from JSON/API
- Display events in date order
- Write basic tests
- Code review

Step 5: Sprint Planning

- Choose a **1–2 week sprint**
- Move selected backlog items into **Sprint 1**
- Assign tasks to team members
- Estimate story points (1, 2, 3, 5, 8)

Step 6: Daily real life industry practice Workflow (Students Follow This Every Day)

1. Check Jira board
2. Move your task from **To Do** → **In Progress**
3. Create a **Git branch** for your task
4. Commit and push code
5. Open a **Pull Request**
6. Link PR to Jira issue
7. After review, merge into dev
8. Move Jira issue to **Done**

Step 7: Sprint Review & Retrospective

At the end of each sprint:

- Demo the working features
- Show the Jira board
- Discuss:
 - What went well
 - What was difficult
 - What to improve next sprint

Git & GitHub Workflow (Clear Steps)

Every task follows this pattern:

1. Create a branch

feature/EPIC1-event-list
bugfix/EPIC2-date-format

2. Write code

Commit frequently.

3. Push branch

Branch appears on GitHub.

4. Open Pull Request

Include:

- Jira issue link
- Description
- Screenshots

5. Code Review

At least one teammate approves.

6. Merge into dev

Never directly into main.

7. Deployment

Cloud platform auto-deploys dev or main.

7. Cloud Deployment (Clear Instructions)

Option A: Netlify (Static or JS Apps)

- Connect GitHub repo
- Select branch (main)
- Auto-deploy on push

Option B: Vercel (React/Next.js)

- Import GitHub repo
- Choose framework
- Deploy

Option C: Render (Node/Flask)

- Create Web Service
- Connect GitHub
- Free tier works well

8. Final Deliverables

Each team submits:

- Link to **GitHub repo**
- Link to **live deployed app**
- Screenshot of **Jira board**
- Short **team reflection**
- Optional: 2-minute demo video